

✓ A. Tiền xử lý dữ liệu

✓ 1. Mount Drive

```
from google.colab import drive
drive.mount('/content/drive')

DATASET_DIR = "/content/drive/MyDrive/HQC MÁY/train_data/images"
```

Mounted at /content/drive

✓ 2. Cài đặt thư viện

```
!pip install pillow imagehash tqdm opencv-python pandas
```

```
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Collecting imagehash
  Downloading ImageHash-4.3.2-py2.py3-none-any.whl.metadata (8.4 kB)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (4.67.3)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.12/dist-packages (from imagehash) (1.9.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from imagehash) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from imagehash) (1.16.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
Downloading ImageHash-4.3.2-py2.py3-none-any.whl (296 kB)
296.7/296.7 kB 6.9 MB/s eta 0:00:00
Installing collected packages: imagehash
Successfully installed imagehash-4.3.2
```

✓ 3. Quét toàn bộ ảnh

```
import os

image_files = []

for root, dirs, files in os.walk(DATASET_DIR):

    for file in files:

        if file.lower().endswith(
            ".jpg", ".jpeg", ".png", ".webp"
        ):
            image_files.append(
                os.path.join(root, file)
            )

print("Total images:", len(image_files))
```

Total images: 1999

✓ 4. Kiểm tra ảnh lỗi

```
from PIL import Image
from tqdm import tqdm

corrupted = []

for path in tqdm(image_files):

    try:
        img = Image.open(path)
        img.verify()

    except:
        corrupted.append(path)
```

```
print("Corrupted:", len(corrupted))
```

```
100%|██████████| 1999/1999 [01:53<00:00, 17.54it/s]Corrupted: 0
```

Sử dụng hàm `verify()` để kiểm tra ảnh bị lỗi:

- Đọc toàn bộ cấu trúc file
- Kiểm tra header
- Kiểm tra dữ liệu nén
- Kiểm tra file có bị cắt giữa chừng không

5. Kiểm tra kích thước ảnh

```
from PIL import Image

small_images = []

for path in tqdm(image_files):

    try:

        img = Image.open(path)

        w, h = img.size

        if w < 150 or h < 200:

            small_images.append(
                (path, w, h)
            )

    except:
        pass

print("Small images:", len(small_images))
```

```
100%|██████████| 1999/1999 [00:09<00:00, 216.83it/s]Small images: 0
```

Nếu chiều rộng < 150 pixel HOẶC chiều cao < 200 pixel

Thì được xem là ảnh nhỏ

6. Phát hiện ảnh mờ, chất lượng kém

```
import cv2
import numpy as np

blurred = []

for path in tqdm(image_files):

    try:

        img = cv2.imread(path)

        gray = cv2.cvtColor(
            img,
            cv2.COLOR_BGR2GRAY
        )

        score = cv2.Laplacian(
            gray,
            cv2.CV_64F
        ).var()

        if score < 100:

            blurred.append(
                (path, score)
            )

    except:
        pass
```

```
print("Blurred:", len(blurred))
```

```
100%|██████████| 1999/1999 [02:47<00:00, 11.96it/s]Blurred: 19
```

Sử dụng thư viện CV để đọc ảnh và đánh giá độ mờ:

- Kỹ thuật Variance of Laplacian: Được sử dụng rất phổ biến để đo độ mờ.
- Sau đó đánh giá bức ảnh theo giá trị score

7. Tìm ảnh trùng bằng pHash

```
from PIL import Image
import imagehash

hash_map = {}

duplicates = []

for path in tqdm(image_files):

    try:

        img = Image.open(path)

        phash = str(
            imagehash.phash(img)
        )

        if phash in hash_map:

            duplicates.append(
                (path, hash_map[phash])
            )

        else:

            hash_map[phash] = path

    except:
        pass

print("Duplicates:", len(duplicates))
```

```
100%|██████████| 1999/1999 [01:27<00:00, 22.73it/s]Duplicates: 5
```

Sử dụng thuật toán Perceptual Hash để phát hiện ảnh trùng lặp: Ý tưởng ở đây là lấy ra giá trị hash của mỗi bức ảnh, các bức ảnh nhìn giống nhau sẽ có giá trị hash (gần) giống nhau.

B. Trực quan hóa dữ liệu

8. Thống kê dataset

```
from PIL import Image
import pandas as pd

stats = []

for path in tqdm(image_files):

    try:

        img = Image.open(path)

        w, h = img.size

        stats.append(
            {
                "file": path,
                "width": w,
                "height": h,
                "ratio": round(w / h, 3)
            }
        )
```

```

    )
)

except:
    pass

df = pd.DataFrame(stats)

df.describe()

```

```

100%|██████████| 1999/1999 [00:07<00:00, 265.70it/s]

```

	width	height	ratio
count	1999.000000	1999.000000	1999.000000
mean	1047.482741	1181.509255	0.924258
std	498.963531	653.749516	0.150005
min	450.000000	500.000000	0.582000
25%	801.500000	900.000000	0.843500
50%	900.000000	900.000000	1.000000
75%	1200.000000	1200.000000	1.000000
max	4968.000000	5456.000000	2.002000

9. Vẽ phân bố kích thước

```

import matplotlib.pyplot as plt

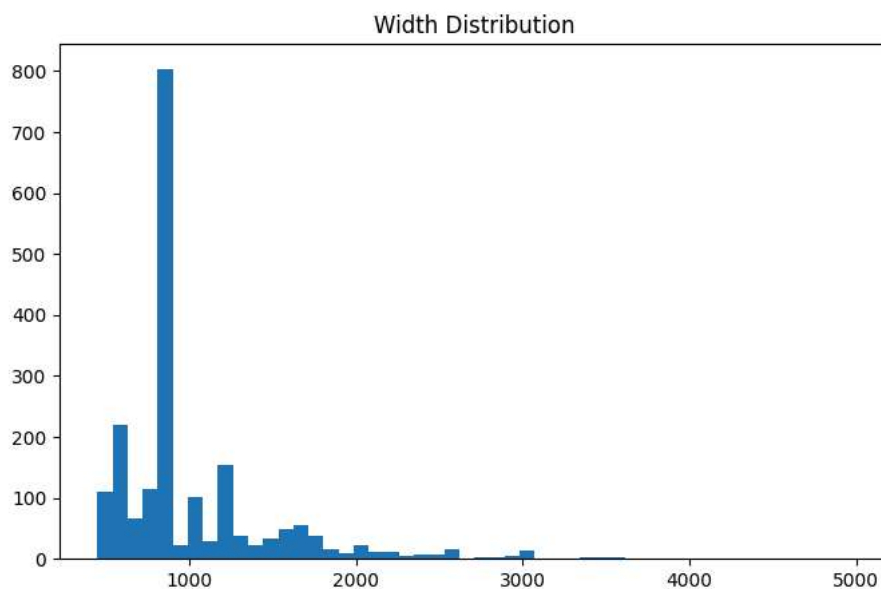
plt.figure(figsize=(8,5))

plt.hist(
    df["width"],
    bins=50
)

plt.title("Width Distribution")

plt.show()

```



- Phân phối lệch phải (Skewed Right): Phần lớn dữ liệu tập trung ở khoảng giá trị nhỏ (dưới 1500), và có một "đuôi" dài kéo về phía các giá trị lớn hơn (lên đến 5000).
- Giá trị phổ biến nhất (Mode): Có một cột cao vượt trội ở khoảng giá trị gần 800 - 900, cho thấy đây là kích thước chiều rộng phổ biến nhất trong tập dữ liệu.
- Mật độ: Đa số các điểm dữ liệu nằm trong khoảng từ 500 đến 2000. Các giá trị trên 3000 xuất hiện rất ít (ngoại lai - outliers).

```

plt.figure(figsize=(8,5))

plt.hist(
    df["height"],

```

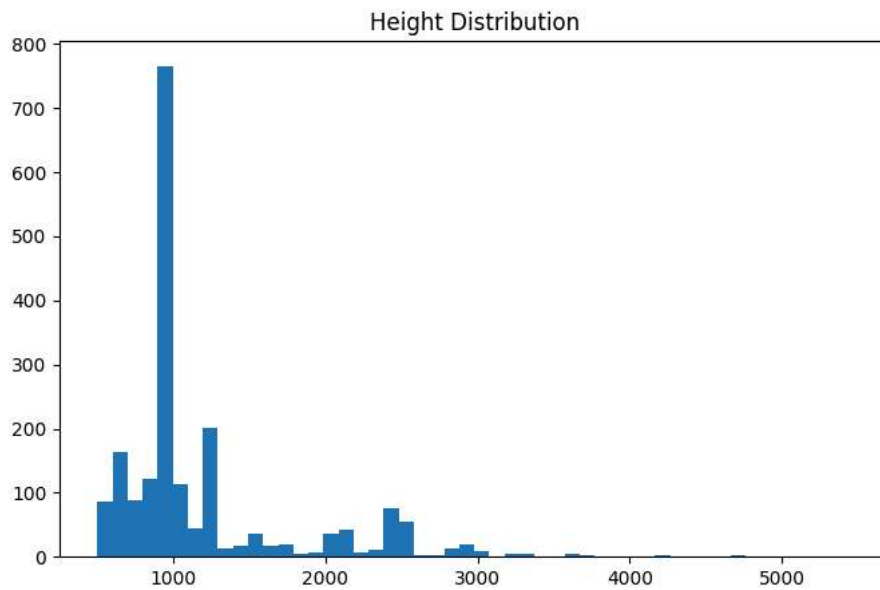
```

bins=50
)

plt.title("Height Distribution")

plt.show()

```



Phân bố chiều cao của tập dữ liệu nhìn chung cũng giống với chiều rộng

10. Tỷ lệ khung hình

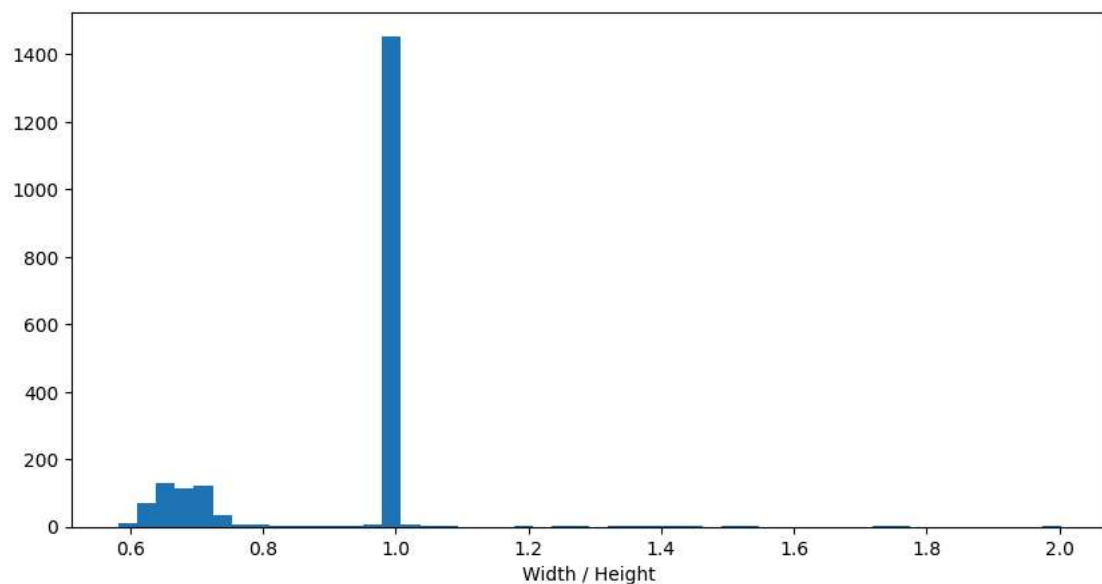
```

plt.figure(figsize=(10,5))

plt.hist(
    df["ratio"],
    bins=50
)

plt.xlabel("Width / Height")
plt.show()

```



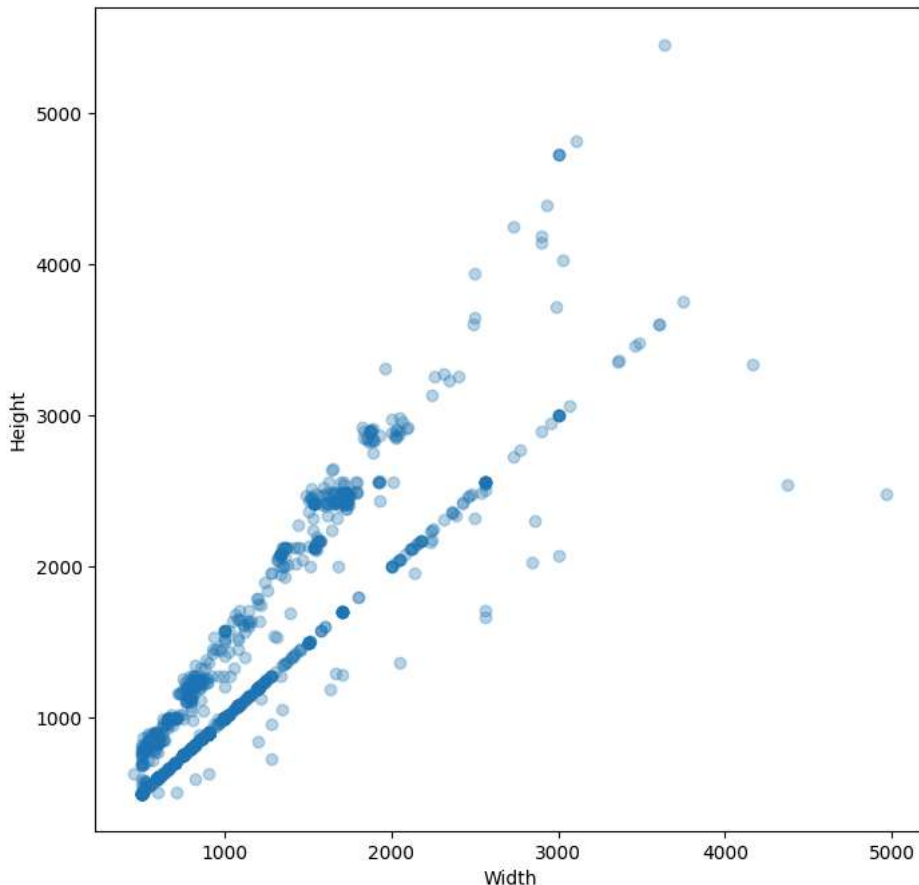
- Tỷ lệ phổ biến nhất: Có một cột cao vượt trội tại giá trị 1.0. Điều này cho thấy phần lớn hình ảnh trong tập dữ liệu có dạng hình vuông (Chiều rộng = Chiều cao).
- Nhóm tỷ lệ nhỏ (0.6 - 0.8): Có một cụm hình ảnh có tỷ lệ khung hình nằm trong khoảng từ 0.6 đến 0.8. Đây thường là các ảnh dạng đứng (portrait).
- Các giá trị ngoại lai: Có một vài hình ảnh rải rác với tỷ lệ lớn hơn 1.2 (ảnh nằm ngang/landscape), nhưng số lượng rất ít không đáng kể.

11. Scatter Plot kích thước

```
plt.figure(figsize=(8,8))

plt.scatter(
    df["width"],
    df["height"],
    alpha=0.3
)

plt.xlabel("Width")
plt.ylabel("Height")
plt.show()
```



- **Mối tương quan thuận mạnh:** Các điểm dữ liệu tập trung chủ yếu dọc theo các đường chéo, cho thấy khi chiều rộng tăng thì chiều cao cũng có xu hướng tăng theo.
- **Các tỉ lệ khung hình phổ biến:** Đường chéo chính (giữa) đại diện cho các hình ảnh hình vuông ($\text{Width} = \text{Height}$). Các đường chéo phụ khác cho thấy các tập hợp hình ảnh có cùng tỉ lệ khung hình chuẩn (như 4:3 hoặc 16:9).
- **Mật độ dữ liệu:** Dữ liệu tập trung dày đặc nhất ở khoảng kích thước nhỏ (dưới 2000px). Giá trị ngoại lai (Outliers): Có một số ít điểm dữ liệu có kích thước rất lớn (trên 4000px - 5000px) nằm rải rác phía trên bên phải.

12. Thống kê định dạng ảnh

```
df["format"].value_counts()
```

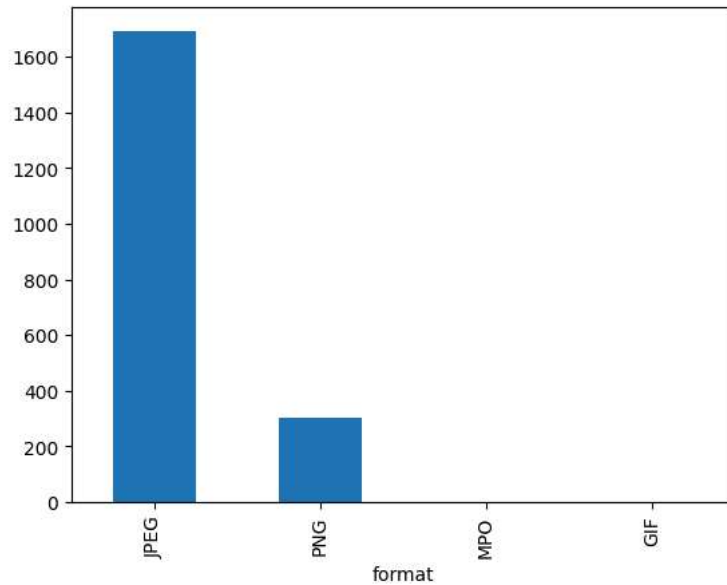
	count
format	
JPEG	1693
PNG	304
MPO	1
GIF	1

dtype: int64

```
formats = df["format"].value_counts()

formats.plot(kind="bar")
```

<Axes: xlabel='format'>



✓ 13. Xuất báo cáo CSV

```
import pandas as pd

pd.DataFrame(
    corrupted,
    columns=["file"]
).to_csv(
    "corrupted.csv",
    index=False
)

pd.DataFrame(
    small_images,
    columns=[
        "file",
        "width",
        "height"
    ]
).to_csv(
    "small_images.csv",
    index=False
)

pd.DataFrame(
    blurred,
    columns=[
        "file",
        "blur_score"
    ]
).to_csv(
    "blurred.csv",
    index=False
)

pd.DataFrame(
    duplicates,
    columns=[
        "duplicate",
        "original"
    ]
).to_csv(
    "duplicates.csv",
    index=False
)
```

✓ 14. Thống kê ảnh bị lỗi

```
blurred_df = pd.read_csv("blurred.csv")
corrupted_df = pd.read_csv("corrupted.csv")
duplicates_df = pd.read_csv("duplicates.csv")
small_df = pd.read_csv("small_images.csv")
```

```
print("Blurred:", len(blurred_df))
print("Corrupted:", len(corrupted_df))
print("Duplicates:", len(duplicates_df))
print("Small Images:", len(small_df))
```

```
Blurred: 19
Corrupted: 0
Duplicates: 5
Small Images: 0
```

Tổng hợp thành bảng thống kê và Trực quan

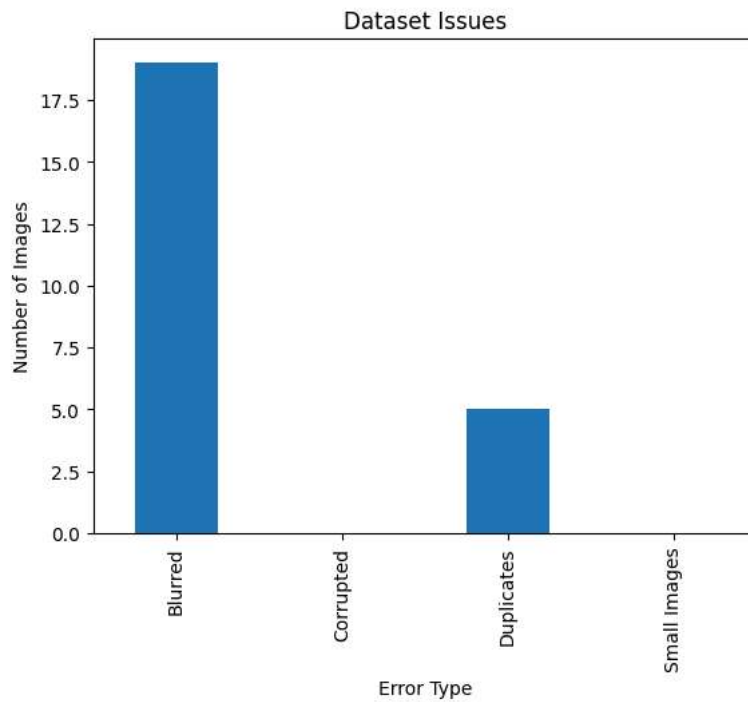
```
summary = pd.DataFrame({
    "Error Type": [
        "Blurred",
        "Corrupted",
        "Duplicates",
        "Small Images"
    ],
    "Count": [
        len(blurred_df),
        len(corrupted_df),
        len(duplicates_df),
        len(small_df)
    ]
})
```

summary

	Error Type	Count
0	Blurred	19
1	Corrupted	0
2	Duplicates	5
3	Small Images	0

```
summary.plot(
    x="Error Type",
    y="Count",
    kind="bar",
    legend=False
)

plt.ylabel("Number of Images")
plt.title("Dataset Issues")
plt.show()
```

Kiểm tra có bị đánh dấu nhầm hay không

```
from PIL import Image
import matplotlib.pyplot as plt

img = Image.open(blurred_df.iloc[0]["file"])

plt.imshow(img)
plt.axis("off")
plt.show()
```



```
row = duplicates_df.iloc[0]

img1 = Image.open(row["duplicate"])
img2 = Image.open(row["original"])

fig, ax = plt.subplots(1, 2, figsize=(10,5))

ax[0].imshow(img1)
ax[0].set_title("Duplicate")

ax[1].imshow(img2)
ax[1].set_title("Original")

plt.show()
```



11. Tự động dọn dẹp

```
import shutil
import os

BAD_DIR = "/content/bad_images"

os.makedirs(
    BAD_DIR,
    exist_ok=True
)
```